

Beyond Representation: Index Structure Limitations in Dense Retrieval

Alessandro Magnani

9th Int. Conf. on Machine Learning and Machine Intelligence
MLMI 2026 | Tokyo, Japan

July 19, 2026

Why this talk: can we drop the sparse side?

Production retrieval today is hybrid: a dense embedding index (HNSW / IVF) *plus* an inverted index (BM25). Two stores, two scoring paths, extra ops.

The tempting question: could a pure-dense stack replace hybrid? Simpler, one index, one score.

Two things have to hold:

- ▶ **Bottleneck 1 — representation.** Can the embedding *encode* what BM25 encodes? Well-studied: Tay et al. 2020, Fan et al. 2022 — capacity grows with d .
- ▶ **Bottleneck 2 — indexing.** Even if the embedding is capable, can an *ANN index* still find it? **This talk. Largely unstudied.**

Headline (the index alone — same embedding)

Flat → **IVF** (partition index the theory targets): forfeits $\sim 33\%$ of achievable Recall@10 on identifier queries, $< 5\%$ on standard.

Flat → **HNSW** (graph index, same population): $\sim 63\%$ on identifier queries, $< 2\%$ on standard.

The population that pays the bill: “spear-fishing” queries

Working definition

Queries whose relevance hinges on a single, low-frequency, high-IDF token: identifiers (SKUs, part numbers), rare terms, precise named entities.

- ▶ Small fraction of traffic, but operationally critical: failed spear-fishing $\Rightarrow$ empty result pages, abandoned carts, broken RAG lookups.
- ▶ BM25 handles them trivially (exact token match).
- ▶ If pure dense is going to replace hybrid, it has to hold up *here*.
- ▶ Our three query subsets: **standard** (control), **rare** (≤ 5 docs), **ID** (extreme case: doc identifier as query).

The idealized case: BM25 is embedding search

Reframe BM25 as a dense inner product in a huge, discrete embedding space $\mathbb{R}^V$ — *one dimension per token* (V = vocabulary size; for our char-trigrams, order 10^6 – 10^7).

Let M = number of docs in the corpus and n_t = number of docs containing token t . Each doc d becomes a sparse, IDF-weighted vector:

$$f(d)_t = \alpha_t = \log(M/n_t) \quad \text{if } t \in d, \text{ else } 0.$$

A query is a *binary indicator*: $q_t = 1$ if $t \in q$, else 0.

For a query $q = \{t^*\}$ with rare token t^* :

- ▶ Relevant doc d_{rel} (contains t^*): $\langle q, f(d_{\text{rel}}) \rangle = \alpha_{t^*}$ — **big**, because n_{t^*} is small.
- ▶ Irrelevant doc: $\langle q, f(d) \rangle = 0$.

Perfect separation on spear-fishing. That is why BM25 works — it is nearest-neighbor search in $\mathbb{R}^V$.

Catch: V is enormous. BM25 handles it with an inverted list — ANN cannot. To use ANN we must compress $\mathbb{R}^V \rightarrow \mathbb{R}^k$ with $k \ll V$. *What compression costs is the rest of the talk.*

Compressing $\mathbb{R}^V \rightarrow \mathbb{R}^k$: Johnson–Lindenstrauss

Random linear map $A : \mathbb{R}^V \rightarrow \mathbb{R}^k$, target dimension $k \in \{256, 512, 1024\}$ (our embedding size — distinct from the document symbol d).

JL theorem (informal)

Given M vectors in $\mathbb{R}^V$, a random projection A preserves all pairwise inner products *up to a multiplicative error* ϵ with high probability, provided

$$k \gtrsim \frac{\log M}{\epsilon^2}, \quad \text{equivalently} \quad \epsilon \approx \sqrt{\frac{\log M}{k}}.$$

What survives, what dies:

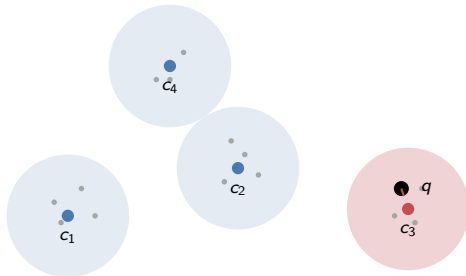
- ▶ Inner products with magnitude $\gg \epsilon$ are preserved. $\Rightarrow$ Spear-fishing signal α_{t^*} from the previous slide easily clears ϵ : **Flat search on the projected embedding is a clean oracle.**
- ▶ Magnitudes $\lesssim \epsilon$ are drowned in projection noise — indistinguishable from random.
- ▶ Bigger k shrinks ϵ ; nothing else does.

So after JL alone, spear-fishing still works. The failure emerges when we add the *second* compression — clustering — on top. IVF.

Quick primer: what is an IVF index?

IVF (Inverted File) — the standard partition-based ANN index.

1. Cluster the corpus into N cells (k-means). Each cell has a *centroid* c .
2. At query time: score the query against all N centroids; keep the top n_{probe} .
3. Only search documents inside those n_{probe} cells.



Speedup: N/n_{probe} fewer distance computations. **Failure mode:** if the relevant cell's centroid is not in the top n_{probe} by query similarity, its documents are *invisible*.

The mechanism: centroid dilution (intuition)

A centroid is the *average* of the vectors in its cell.

- ▶ A *common* token appears in many docs $\Rightarrow$ contributes strongly to the average $\Rightarrow$ shows up in the centroid.
- ▶ A *rare* token appears in a handful of docs $\Rightarrow$ its contribution is averaged away $\Rightarrow$ nearly invisible in the centroid.

Now consider a query dominated by that rare token:

- ▶ Query vector is nonzero on the rare-token direction.
- ▶ Every centroid — *including the one containing the relevant docs* — has *almost zero* weight on that direction (averaged away).
- ▶ Result: **all centroids look equally (un)related to the query**. Cluster selection becomes essentially random.

The failure is not the embedding. It is the averaging step in the index.

The bound: when does the centroid signal survive?

So far: in $\mathbb{R}^V$ the relevant doc scores α_{t^*} ; JL preserves it. But **IVF selects clusters by the centroid**, and a centroid is an average.

Step 1 — Centroid. Partition into N clusters, average size $|C| \approx M/N$: $c_C = \frac{1}{|C|} \sum_{d \in C} f(d)$.

Step 2 — Prevalence. Let $n_{t^*} = \#$ docs in C containing t^* . Only these contribute to the t^* -direction of c_C :

$$c_C|_{t^*} \approx \underbrace{\frac{n_{t^*}}{|C|}}_p \cdot \alpha_{t^*}.$$

$p = \text{prevalence of } t^* \text{ in } C$

Rare token $\Rightarrow p$ tiny (often $1/|C|$, or 0).

Step 3 — Query. q = binary indicator of t^* ($q_{t^*} = 1$):

$$\langle q, c_C \rangle \approx \alpha_{t^*} \cdot p$$

Survival condition

Relevant centroid picked *only when* $\alpha_{t^*} \cdot p \gg \epsilon$ (JL floor). Rare: α big, p tiny $\Rightarrow$ product $< \epsilon \Rightarrow$ wrong cluster. Bigger k helps ϵ ; nothing helps p .

HNSW isn't a way out

HNSW is not partition-based — it is a *layered proximity graph*. Greedy descent from a hub node down to the query's local neighborhood.

So why does it fail on the same queries?

- ▶ Graph edges are built from Euclidean distances during insertion.
- ▶ Dense regions of the embedding space get thick, well-connected subgraphs. Rare vectors land in sparse regions — *few incoming edges from any hub*.
- ▶ Greedy search from a hub cannot reach an orphan node without a long detour: it stops at a locally-good but globally-wrong region.
- ▶ Widening the beam (`efSearch`) helps — but with diminishing returns.

The pattern is the same: rare-token vectors are structurally hard to *reach*, whether the structure is a partition (IVF) or a graph (HNSW). Different mechanism; **same victims**.

Experimental setup (compact)

Datasets (1,000 queries per subset)

- ▶ Home Depot (HD): 54,667 product titles
- ▶ MS MARCO Passage v1.1: 82,360 passages

Query subsets

- ▶ standard | rare (≤ 5 docs) | ID

Representation: char-trigram + JL projection, $d \in \{256, 512, 1024\}$, unsupervised.

Indices (FAISS): Flat, IVF, HNSW ($M=16$, efC=200), BM25 baseline (Pyserini).

Why unsupervised trigrams?

A supervised encoder abstracts away the lexical detail that *defines* spear-fishing queries. To isolate the *index* bottleneck from the representation bottleneck we need a representation that provably preserves lexical distances (Johnson–Lindenstrauss).

Headline result: the Flat $\rightarrow$ HNSW gap

Dataset	Subset	Flat	IVF	HNSW	BM25
HD	standard	0.327	0.314	0.321	0.447
HD	rare	0.751	0.642	0.619	0.878
HD	id	0.874	0.590	0.327	1.000
MS MARCO	standard	0.471	0.448	0.446	0.770
MS MARCO	rare	0.205	0.187	0.166	0.688
MS MARCO	id	0.174	0.152	0.071	—

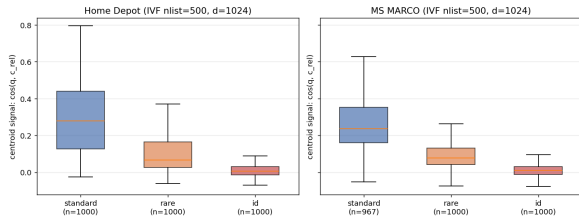
Recall@10 at $d=1024$, best ANN sweep config.

- ▶ Relative Flat $\rightarrow$ HNSW loss: HD {standard **1.8%**, rare **17.5%**, **id 62.6%**}; MM analogous.
- ▶ The gap tracks the query population exactly as the theory predicts.

Direct evidence: measure the centroid signal

For each query: compute $\langle q, c_{\text{rel}} \rangle$ on the centroid of the query's *relevant* cluster (IVF, $n_{\text{list}}=500$, $d=1024$, HD).

Centroid signal for relevant cluster, by query subset

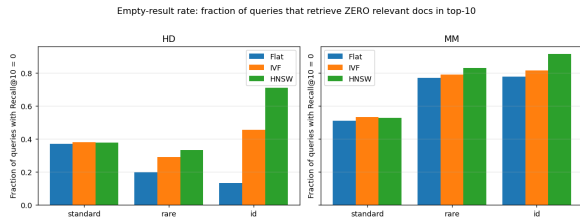


Subset	signal	rank	hit@250
standard	0.279	2	97%
rare	0.066	69	81%
id	0.007	216	59%

$\sim 40\times$ collapse of the centroid signal from standard to ID — the correct cluster is buried at rank ~ 200 .

The bound $\langle c, q \rangle \approx \alpha_{t^*} \cdot p$ is not just suggestive — it is what we *measure*.

The production-visible symptom



Fraction of queries with zero relevant docs in top-10:

- ▶ HD-id: Flat **13%** → HNSW **71%**
- ▶ MM-id: Flat **78%** → HNSW **92%**
- ▶ Standard: within ± 2 pp on both datasets.

Cost to match Flat recall (efSearch, HD):

- ▶ standard: **2.7 $\times$** faster than Flat
- ▶ id: **40 $\times$** slower than Flat

ANN's speedup evaporates on the population that needs it most.

“But learned encoders would fix this”

Anticipated question: *do supervised dense encoders escape the issue?*

Re-ran with all-MiniLM-L6-v2 (Sentence-BERT, 384-dim):

Dataset	Subset	Trigram Flat R@10	MiniLM Flat R@10
HD	standard	0.327	0.298
HD	rare	0.751	0.428
HD	id	0.874	0.010
MS MARCO	standard	0.471	0.894
MS MARCO	id	0.174	0.001

- ▶ MiniLM *cannot distinguish identifiers* — “12345” $\approx$ “67890”.
- ▶ Representation collapses *before* the index has a chance to bias.
- ▶ Which is precisely why hybrid retrieval exists.

Takeaways

1. Production dense retrieval is hybrid for a reason. If you want to drop the sparse side, you have to clear *two* bottlenecks, not one.
2. Bottleneck 1 (representation) is well-studied. Bottleneck 2 (indexing) is what this paper isolates and quantifies.
3. Centroid dilution + JL noise floor give a closed-form bound; the $\sim 40\times$ centroid-signal collapse from standard to ID is exactly what the bound predicts.
4. HNSW is not a workaround — the mechanism differs, the victims don't.
5. Practical implication: query-type-aware routing, index-aware training, or — most simply — keep the sparse side.

Code & experiments: github.com/alemagnani/sparse-dense-paper

Thank you

Questions?

Alessandro Magnani | ale.magnani@gmail.com

MLMI 2026 | Tokyo, Japan | July 19, 2026